



Red Hat OpenShift AI Self-Managed 3.4

Experimenting with models in the gen AI playground

Experiment with models in the gen AI playground in Red Hat OpenShift AI Self-Managed

Red Hat OpenShift AI Self-Managed 3.4 Experimenting with models in the gen AI playground

Experiment with models in the gen AI playground in Red Hat OpenShift AI Self-Managed

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

As a Red Hat OpenShift AI user, you can experiment with models in the gen AI playground in Red Hat OpenShift AI Self-Managed.

Table of Contents

PREFACE	3
CHAPTER 1. PLAYGROUND OVERVIEW	4
CHAPTER 2. PLAYGROUND PREREQUISITES	5
2.1. CLUSTER ADMINISTRATOR PREREQUISITES	5
2.2. USER PREREQUISITES	5
2.3. CONFIGURING MODEL CONTEXT PROTOCOL (MCP) SERVERS	5
2.4. MODEL AND RUNTIME REQUIREMENTS FOR THE PLAYGROUND	6
2.4.1. Key model selection factors	7
2.4.2. Example model configuration	7
2.5. ABOUT THE AI ASSETS ENDPOINTS PAGE	8
CHAPTER 3. CONFIGURE A PLAYGROUND FOR YOUR PROJECT	10
CHAPTER 4. ENABLE CUSTOM ENDPOINTS FOR THE PLAYGROUND	11
CHAPTER 5. CREATE AND USE CUSTOM ENDPOINTS IN THE PLAYGROUND	13
CHAPTER 6. HOW MODEL EXPERIMENTATION ACCELERATES MODEL SELECTION	15
6.1. TEST MODEL RESPONSES	16
CHAPTER 7. TEST YOUR MODEL WITH RETRIEVAL AUGMENTED GENERATION (RAG)	18
CHAPTER 8. RAG SETTINGS	20
CHAPTER 9. REUSABLE SYSTEM INSTRUCTIONS	21
9.1. SAVE SYSTEM INSTRUCTIONS AS A REUSABLE PROMPT	21
9.2. REUSE A SAVED PROMPT IN A PLAYGROUND SESSION	22
CHAPTER 10. TESTING WITH MODEL CONTEXT PROTOCOL (MCP) SERVERS	24
CHAPTER 11. EXPORTING YOUR PLAYGROUND CONFIGURATION	25
CHAPTER 12. UPDATING YOUR PLAYGROUND CONFIGURATION	26
CHAPTER 13. DELETING A PLAYGROUND FROM YOUR PROJECT	27
CHAPTER 14. NEXT STEPS	28
CHAPTER 15. TROUBLESHOOTING PLAYGROUND ISSUES	29
15.1. THE CHATBOT THINKS INDEFINITELY	29
15.2. THE MODEL DOES NOT USE RAG DATA	29
15.3. MCP SERVERS ARE MISSING FROM THE UI	29
15.4. THE MODEL FAILS TO CALL MCP TOOLS	29

PREFACE

Use the generative AI (gen AI) feature in OpenShift AI to evaluate, test, and interact with foundation and custom models in your project. You can test prompt engineering with Retrieval-Augmented Generation (RAG) and validate model behavior before using the model in an application.

CHAPTER 1. PLAYGROUND OVERVIEW



IMPORTANT

Gen AI playground is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

The generative AI (gen AI) playground is an interactive environment within the Red Hat OpenShift AI dashboard where you can prototype and evaluate foundation models, custom models, and Model Context Protocol (MCP) servers before you use them in an application.

You can test different configurations, including retrieval augmented generation (RAG), to determine the right assets for your use case. After you find an effective configuration, you can retrieve a Python template that serves as a starting point for building and iterating in a local development environment.

The playground interface consists of a chat area and a settings panel on the left side. The settings panel is organized into tabs:

- **Model:** Select a model and adjust parameters such as temperature and streaming.
- **Prompt:** Write, save, and load system instructions.
- **Knowledge:** Upload files for RAG or select an external vector store.
- **MCP:** Connect to Model Context Protocol servers and authorize access to their tools.



NOTE

Chat history and parameter settings are not preserved if you refresh your browser or end your session. However, prompts that you save through the **Prompt** tab are stored in MLflow and persist across sessions.

The playground supports the following workflows:

- Chat with foundation models and custom-deployed models, including models from external endpoints.
- Test prompt engineering with RAG by uploading documents or selecting external vector stores as knowledge sources.
- Compare model responses side by side by opening multiple chat panes.
- Save and version system instructions as reusable prompts.
- Authorize and interact with MCP servers and their tools.
- Export your playground configuration as a Python code template.

CHAPTER 2. PLAYGROUND PREREQUISITES

Before you can configure and use the gen AI playground feature, you must meet prerequisites at both the cluster and user levels.

2.1. CLUSTER ADMINISTRATOR PREREQUISITES

Before a user can configure a playground instance, a cluster administrator must complete the following setup tasks:

- Ensure that OpenShift AI is installed on an OpenShift cluster running version 4.19 or later.
- Set the value of the **spec.dashboardConfig.genAiStudio** dashboard configuration option to **true**. For more information, see [Dashboard configuration options](#).
- If using OpenShift AI groups, add users to the **rhods-users** and **rhods-admins** OpenShift group.
- Ensure that the Llama Stack Operator is enabled on the OpenShift cluster by setting its **managementState** field to **Managed** in the **DataScienceCluster** custom resource (CR) of the OpenShift AI Operator. For more information, see [Activating the Llama Stack Operator](#).
- Configure the Model Context Protocol (MCP) servers to test models with external tools. For more information, see [Configuring model context protocol servers](#).

2.2. USER PREREQUISITES

After the cluster administrator completes the setup, you must complete the following tasks before you can configure your playground instance:

- You are logged in to OpenShift AI.
- If you are using OpenShift AI groups, you are a member of the appropriate user or admin group.
- Create a project. The playground instance is tied to a project context. For more information, see [Creating a project](#).
- Add a connection to your project. For more information about creating connections, see [Adding a connection to your project](#).
- Deploy a model in your project and make it available as an AI asset endpoint. For more information, see [Deploying models on the model serving platform](#).

After you complete these tasks, the project is ready for you to configure your playground instance.

2.3. CONFIGURING MODEL CONTEXT PROTOCOL (MCP) SERVERS

A cluster administrator must configure and enable the Model Context Protocol (MCP) servers at the platform level before users can interact with external tools in the Generative AI Playground. This configuration is done by creating a **ConfigMap** in the **redhat-ods-applications** namespace, which holds the necessary information for each MCP server.

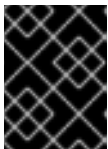
Prerequisites

- You have cluster admin privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform

Procedure

1. Create a file named **gen-ai-aa-mcp-servers.yaml** with the following YAML content. You can add multiple server entries under the **data:** field.

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: gen-ai-aa-mcp-servers
  namespace: redhat-ods-applications
data:
  GitHub-MCP-Server: |
    {
      "url": "https://api.githubcopilot.com/mcp/x/repos/readonly",
      "description": "The GitHub MCP server enables exploration and interaction with
repositories, code, and developer resources on GitHub. It provides programmatic access to
repositories, issues, pull requests, and related project data, allowing automation and
integration within development workflows. With this service, developers can query
repositories, discover project metadata, and streamline code-related tasks through MCP-
compatible tools."
    }
```



IMPORTANT

The **ConfigMap** key (**GitHub-MCP-Server**) is case-sensitive and must be unique. The content provided under this key must be valid JSON format.

2. Apply the **ConfigMap** to the cluster by running the following command:

```
oc apply -f gen-ai-aa-mcp-servers.yaml
```

Verification

- Confirm that the **ConfigMap** was successfully applied by running the following command:

```
oc get configmap gen-ai-aa-mcp-servers -n redhat-ods-applications -o yaml | grep GitHub-MCP-Server
```

- The output should contain the key name, confirming its successful creation:

```
GitHub-MCP-Server: |
```

2.4. MODEL AND RUNTIME REQUIREMENTS FOR THE PLAYGROUND

To successfully use the retrieval augmented generation (RAG) and Model Context Protocol (MCP) features in the playground, the model you deploy must meet specific requirements. Not all models offer the same capabilities.

2.4.1. Key model selection factors

Tool calling capabilities

The model must support tool calling to interact with the playground's RAG and MCP features. You must check the model card (for example, on Hugging Face) to verify this capability. For more information, see [Tool calling](#) in the vLLM documentation.

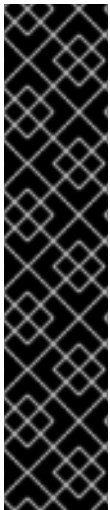
Context length

Models with larger context windows are recommended for RAG applications. A larger context window allows the model to process more retrieved documents and maintain longer conversation histories.

vLLM version and configuration

Tool calling functionality depends heavily on the version of vLLM used in your model serving runtime.

- **Version:** Use the latest vLLM version included in Red Hat OpenShift AI for optimal compatibility.
- **Runtime arguments:** You must configure specific runtime arguments in the model serving runtime to enable tool calling. Common arguments include (not exhaustive):
 - **--enable-auto-tool-choice**
 - **--tool-call-parser**
 - **--chat-template=/opt/app-root/template/<template_file>.jinja**



IMPORTANT

- If these requirements are not met, the model might fail to search documents or execute tools without returning a clear error message.
- Tool calling functionality varies by model family, such as Llama, Mistral, Qwen and so on. For a complete list of supported models, compatible parsers, and template filenames, see [Tool calling](#) in the vLLM documentation.
- When you specify a chat template, use the absolute path **/opt/app-root/template/** to locate the standard Jinja template files provided in the Red Hat OpenShift AI image. For example, **/opt/app-root/template/tool_chat_template_llama3.1_json.jinja**. Do not use relative paths, such as **examples/**. Relative paths cause model deployment to fail.

2.4.2. Example model configuration

The following table describes an example configuration for the **Qwen/Qwen3-14B-AWQ** model for use in the playground. You can use this as a reference when configuring your own model runtime arguments.

Table 2.1. Example configuration for Qwen/Qwen3-14B-AWQ

Field	Configuration Details
Model	Qwen/Qwen3-14B-AWQ
vLLM Runtime	vLLM NVIDIA GPU ServingRuntime for KServe
Hardware Profile	NVIDIA A10G (24GB VRAM)
Custom Runtime Arguments	--dtype=auto --max-model-len=32768 --enable-auto-tool-choice --tool-call-parser=hermes --reasoning-parser=qwen3 --gpu-memory-utilization=0.90

2.5. ABOUT THE AI ASSETS ENDPOINTS PAGE



IMPORTANT

This feature is currently available in Red Hat OpenShift AI 3.4 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

The **AI asset endpoints page** is a central dashboard for managing the generative AI assets available for you to use within your project.

The page organizes assets into the following categories:

- Models: Lists generative AI models available in your project from the following sources:
 - Models deployed in your project namespace that have been designated as available assets. For a model to be available, you must select the **Add as AI asset endpoint** check box when deploying it. For more information, see [Deploying models on the model serving platform](#).
 - Custom endpoints created from models in other namespaces on the same cluster or from external third-party providers. Custom endpoints are available when a platform engineer enables the feature. For more information, see [Create and use custom endpoints in the playground](#).
 - MaaS models provided through the Models as a Service platform.
- Model Context Protocol (MCP) Server: Lists all available MCP servers configured in the cluster in a config map. For more information, see [Configuring Model Context Protocol servers](#).

The primary purpose of this page is to provide a starting point for using these assets. From here, you can perform actions such as adding a model to a playground instance for testing.



IMPORTANT

The assets listed on the **AI assets endpoints** page are scoped to your currently selected project. You only see models and servers that are deployed and available within that specific project.

CHAPTER 3. CONFIGURE A PLAYGROUND FOR YOUR PROJECT

Configure a generative AI (gen AI) playground for your project so that you can interact with your deployed generative AI models and connect to backend servers such as Model Context Protocol (MCP) servers.

Prerequisites

- You have created a project.
- You have deployed a model in your project and added your model as an AI asset endpoint.
- If your cluster administrator has configured Model Context Protocol (MCP) servers, they are accessible within your OpenShift environment.

Procedure

1. Perform one of the following actions:
 - a. From the OpenShift AI dashboard side navigation menu, click **Gen AI studio → Playground**.
 - i. Select the project containing your model deployment from the **Project** drop-down list.
 - ii. Click **Create playground**
The **Configure playground** dialog opens.
 - b. From the OpenShift AI dashboard side navigation menu, click **Gen AI studio → AI asset endpoints**.
 - i. Select the project containing your model deployment from the **Project** drop-down list.
 - ii. Click the **Models** tab.
 - iii. Locate the model that you want to create a playground for, and then click **Add to playground**.
The **Configure playground** dialog opens.
2. Select the models that you want to use in this playground instance. For each model, use the **Type** dropdown to select **Inference** or **Embedding**.
3. Click **Create**.
Wait for the playground interface to finish loading.

Verification

- The playground interface loads successfully with a chat area and a **Settings** panel on the left side.
- In the **Settings** panel, the **Model** tab displays the selected model.
- The **Model** list in the chatbot header bar shows the name of your selected model.

CHAPTER 4. ENABLE CUSTOM ENDPOINTS FOR THE PLAYGROUND



IMPORTANT

Custom endpoints is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

As a platform engineer or cluster administrator, you can enable the custom endpoints feature to let users create endpoints from models that are not deployed in their project namespace. The following dashboard configuration options in the **Odhdashboardconfig** custom resource control this feature:

- **aiAssetCustomEndpoints** controls whether users can see and use the custom endpoints feature. When enabled, users can create endpoints from models deployed in another namespace on the same cluster.
- **externalProviders** controls whether users can also create endpoints for external third-party providers such as OpenAI, Anthropic, or AWS. When enabled, data from the responses API can be sent outside the cluster.
- **clusterDomains** specifies additional domains that are considered internal to the cluster.



WARNING

Enabling external providers allows users to send data from the responses API, including RAG context, MCP tool results, and user input, to endpoints outside the cluster. Evaluate your organization's data security requirements before enabling this flag.

Prerequisites

- You have cluster administrator privileges in OpenShift.
- You have access to the **Odhdashboardconfig** custom resource.

Procedure

1. Log in to OpenShift as a cluster administrator.
2. Open the **Odhdashboardconfig** custom resource for editing.
For more information, see [Editing the dashboard configuration](#).
3. In the **spec** section, set the following configuration options:

```
spec:
  dashboardConfig:
    aiAssetCustomEndpoints: true
  genAiStudioConfig:
    aiAssetCustomEndpoints:
      externalProviders: true
    clusterDomains: []
```

where:

aiAssetCustomEndpoints

Specifies whether to show the custom endpoints feature. Set to **true** to allow users to create custom model endpoints. The default value is **false**.

externalProviders

Specifies whether users can create external third-party provider endpoints. Set to **true** to enable external providers. The default value is **false**. Requires **aiAssetCustomEndpoints** to also be set to **true**.

clusterDomains

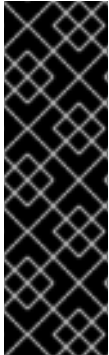
Specifies an array of domains that are considered internal to the cluster, in addition to **.svc.cluster.local**, which is always treated as internal.

1. Save the custom resource.

Verification

- Users can see the **Create endpoint** option on the **AI asset endpoints** page.
- If you enabled external providers, users can create endpoints for external third-party providers.

CHAPTER 5. CREATE AND USE CUSTOM ENDPOINTS IN THE PLAYGROUND



IMPORTANT

Custom endpoints is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

You can create custom endpoints to use models that are not deployed in your project namespace. Custom endpoints let you access models deployed in another namespace on the same cluster, or models hosted by external third-party providers such as OpenAI, Anthropic, or AWS.

After you create a custom endpoint, the model appears on the **AI asset endpoints** page and is available for selection in the playground.



WARNING

When you connect to an external third-party provider, data from the responses API, including RAG context, MCP tool results, and user input, is sent outside your cluster. Ensure that your organization's security policies allow this before creating an external endpoint.

Prerequisites

- You have configured a playground for your project.
- Your platform engineer has enabled the custom endpoints feature for your project. For more information, see [Enable custom endpoints for the playground](#).
- You can create configmaps and secrets in your namespace.
- If you are connecting to a model in another namespace, you have the internal API endpoint URL and an authorized access token.
- If you are connecting to an external provider, you have the provider's API endpoint URL and a valid API key.

Procedure

1. From the OpenShift AI dashboard, click **Gen AI studio** → **AI asset endpoints**.
2. Click **Create endpoint**

3. In the form, enter the following details:

Model type

Specifies whether the model is an inferencing model or an embedding model. Inferencing models generate text responses and are used in the playground. Embedding models convert text to vectors and are used in RAG pipelines.

Model ID

Specifies the exact model identifier from your provider. This must match the provider's model ID exactly.

Display name

Specifies a user-friendly name for the model, shown in tables and selectors instead of the model ID.

Embedding dimension

For embedding models only. Specifies the output vector size for the embedding model.

URL

Specifies the API endpoint for the model service. For a model in another namespace, use the internal cluster URL, for example **https://<service-name>.<namespace>.svc.cluster.local**. For an external provider, use the provider's API URL.

Token

Specifies the API key or access token required to authenticate with the endpoint. The credential is stored as a Kubernetes Secret and is shared at the project level.

Use case

Optional. Specifies what the model is best suited for, to help other users in the project identify the model's purpose.

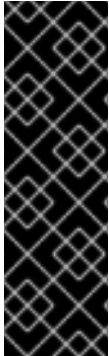
4. Optional: Click **Verify model** to send a test request to the endpoint. The verification checks that the endpoint is reachable and returns an OpenAI-compatible response.
5. Click **Create**.
The custom endpoint appears on the **AI asset endpoints** page.
6. Click **Add to Playground** to make the model available in the playground.
7. To use the model, click **Try in Playground** or select the model from the models dropdown in the playground.

Verification

- The custom endpoint appears on the **AI asset endpoints** page with the correct endpoint type.
- You can select the model in the playground and receive inference responses.

CHAPTER 6. HOW MODEL EXPERIMENTATION ACCELERATES MODEL SELECTION

The playground provides an interactive environment for evaluating how models respond to your prompts. You can test a single model's baseline behavior or compare two models side-by-side to find the best fit for your use case.



IMPORTANT

Multi-model comparison is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

You can adjust the following parameters to see how they affect response quality, tone, and latency:

Temperature

Controls the randomness of the model's output. Use values between 0 and 2. The temperature value directly influences creativity:

- Values near 0: Produce deterministic and factual responses, for objective or factual tasks.
- Values around 0.7: A common default for balanced output.
- Values near 1: Increase creativity and randomness, for generative or creative tasks.
- Values over 1: Typically produce incoherent output.

Streaming

Shows the model's response as it is generated. This is helpful for testing model latency and seeing the model's progress in real time. When streaming is off, the full response does not render until it is complete.

System instructions

Defines the context, persona, or instructions for the model. The playground provides a default prompt that you can review or edit.

When comparing models, multi-model comparison helps you:

- Evaluate multiple models in a single session instead of configuring separate playground instances.
- Compare response quality, tone, accuracy, and latency across models with the same prompt.
- Assess tradeoffs between open source and commercial models, or between different versions of the same model.
- Select the best model for a specific domain or use case.

Additional resources

- [Test model responses](#)
- [About the AI assets endpoints page](#)

6.1. TEST MODEL RESPONSES

Use the playground to test and evaluate your model responses. You can test a single model or compare two models side-by-side.

Prerequisites

- You have configured a playground for your project.
- At least two models are deployed and available as AI asset endpoints in your project if you want to compare models.

Procedure

1. From the OpenShift AI dashboard, click **Gen AI studio → Playground**.
2. From the **Model** list in the chatbot header bar, select the model that you want to test.
3. Adjust model parameters such as temperature, streaming, and system instructions as needed. For more information about these parameters, see [How model experimentation accelerates model selection](#).
4. In the chat input field, type a query.
5. Click **Send**.
The model response appears in the chat area. The response header shows the name of the model that generated the response.



NOTE

After you send a prompt, the **Send** button changes to a **Stop** button. Click it to interrupt the model response.

6. Optional: To clear the chat history and start a new conversation, click **New Chat**. Your playground configuration settings are preserved.
7. Optional: To compare two models side-by-side, click **Compare** in the playground configuration panel.
The playground clears your current chat history and copies your configuration to both chat panels. This action cannot be undone.

The interface expands to display two model response panels.



IMPORTANT

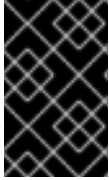
Multi-model comparison is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

- a. In the **Model** dropdown for each comparison panel, select a model.
- b. In the chat input field, enter a prompt to test across both models.
- c. Click **Send**.
The playground displays responses from both models simultaneously in separate panels.
- d. Optional: To remove a model from the comparison, click the **X** in the top-right corner of that model's chat panel.

CHAPTER 7. TEST YOUR MODEL WITH RETRIEVAL AUGMENTED GENERATION (RAG)

You can enhance your model responses by providing it with contextual information from your own documents using retrieval augmented generation (RAG). You can upload documents to the vector database associated with the playground to provide context for your model responses.



IMPORTANT

The RAG feature of the gen AI playground is currently configured to work only with an inline vector database. There is currently no mechanism to configure the playground to connect this RAG feature to an external or remote vector database.

Prerequisites

- You have configured a playground for your project.
- You have the document files ready to upload. The supported file formats are PDF, DOC, or CSV. You can upload up to 10 files, with a maximum size of 10MB per file.

Procedure

1. From the OpenShift AI dashboard, click **Gen AI studio** → **Playground**.
2. In the settings panel, click the **Knowledge** tab.
3. Click **Upload files**.
4. Click **Upload**.
The **Upload files** dialog opens.
5. Drag and drop your file or click to browse and select a file from your local system.
6. Optional: Adjust the **Maximum chunk length** and **Chunk overlap** and **Delimiter** values as needed for your document type. For more information about these settings, see [Understanding RAG settings](#).
7. Click **Upload**.
Wait for the file to finish processing. A **Source uploaded** notification appears, and the file is listed under **Uploaded files**.
8. Repeat these steps to upload additional files if needed.
9. Optional: In the **Prompt** tab, review or edit the system instructions. The playground provides a default prompt.
10. In the chat input field, ask a question related to your documents that the model would not know otherwise.
11. Observe the model response.

TIP

If the model does not use its knowledge search tool, you can modify the system instructions in the **Prompt** tab to change this behavior.

To ensure that the model actively utilizes the available RAG documents rather than relying solely on its pre-trained data, refine the system prompt by including directives as shown in the following examples:

- **To force use:** "You **MUST** use the **knowledge_search** tool to obtain updated information."
- **To specify context:** "Always search the knowledge base before answering questions about company policies, recent events, or specific documentation."



NOTE

After you send a prompt, the **Send** button changes to a **Stop** button. Click it to interrupt the model response.

12. Optional: To clear the chat history and start a new conversation, click **New Chat**. Your playground configuration settings are preserved.

Verification

- The model retrieves information from the uploaded documents to answer the questions.

CHAPTER 8. RAG SETTINGS

When you upload a document for retrieval augmented generation (RAG), you can configure the following settings to optimize how the text is processed.

Maximum chunk length	The maximum word count for each text section ("chunk") created from your uploaded files. • Smaller chunks are recommended for precise data retrieval. • Larger chunks are recommended for tasks requiring broader context, such as summarization.
Chunk overlap	The number of words from the end of one text section (chunk) that are repeated at the start of the next one. This overlap helps maintain continuous context across chunks, improving model responses. For example, the following sentence is chunked differently depending on the chunk overlap: "Chunk overlap can improve the quality of model responses." Maximum chunk length = 4, Chunk overlap = 1 Chunk overlap can improve improve the quality of of model responses. Maximum chunk length = 4, Chunk overlap = 0 Chunk overlap can improve the quality of model responses.
Delimiter	A character or string that specifies where a text chunk should end. This helps define text boundaries alongside maximum chunk length and overlap, ensuring sentences or paragraphs remain intact. Examples of delimiters: • <code>.</code> (period): splits at sentence boundaries • <code>\n</code> (newline): splits at paragraph boundaries • <code>;</code> (semicolon): splits at clause boundaries For example, the following sentence is split as follows depending on the delimiter: "This is the first sentence. This is the second sentence." Maximum chunk length = 4, Chunk overlap = 0 This is the first sentence. This is the second sentence. Maximum chunk length = 4, Chunk overlap = 0, Delimiter = 0 This is the first sentence. This is the second sentence.

CHAPTER 9. REUSABLE SYSTEM INSTRUCTIONS



IMPORTANT

Prompt management is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

When you find system instructions that produce effective results in the playground, you can save them as a named prompt so that you and your team can reuse them across sessions. Each time you refine and save a prompt, a new version is created automatically, so you can track what changed and return to earlier versions if needed.

Saved prompts are stored in MLflow and scoped to your project. Any project member can browse and load prompts that others have saved, making it easier to standardize on instructions that work well for your use case.

You can work with saved prompts from two locations:

Prompt tab in the playground

While testing a model, use the **Prompt** tab in the chatbot settings panel to save your current instructions, load prompts that you or your team have saved, or continue refining a prompt across sessions. The interface tracks your edits and shows an **Unsaved** indicator when your instructions differ from the last saved version.

Prompts page

To review and manage saved prompts outside of a playground session, use the **Prompts** page under **Gen AI studio**. This page provides a standalone view of all saved prompts in your project through the MLflow prompt registry.

9.1. SAVE SYSTEM INSTRUCTIONS AS A REUSABLE PROMPT



IMPORTANT

Prompt management is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

After you write system instructions that produce effective results, you can save them as a named prompt so that you and your team can reuse them. If you later refine the instructions, saving again creates a new version automatically, so you can track your changes over time.

Prerequisites

- You have configured a playground for your project.
- The MLflow service is available in your project.

Procedure

1. From the OpenShift AI dashboard, click **Gen AI studio** → **Playground**.
2. In the chatbot settings panel, click the **Prompt** tab.
3. Write or edit the system instructions in the **Instructions** field.
4. Click **Save**.
The **Save prompt** dialog opens.
5. In the **Name** field, enter a name for the prompt.
6. Optional: In the **Commit message** field, describe your changes.
7. Click **Create**.
A notification confirms that the prompt was saved to your project.

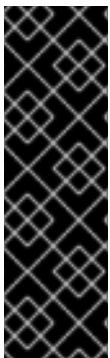
To update a saved prompt and create a new version:

1. Click **Edit** and update the system instructions. An **Unsaved** indicator appears.
2. Click **Save**.
The **New prompt version** dialog opens with the prompt name and the next version number.
3. Optional: In the **Commit message** field, describe what changed in this version.
4. Click **Save**.

Verification

- The prompt name and version number are displayed in the **Prompt** tab.
- The **Unsaved** indicator is no longer visible.

9.2. REUSE A SAVED PROMPT IN A PLAYGROUND SESSION



IMPORTANT

Prompt management is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

If you or a team member have saved system instructions as a prompt, you can load them into a new playground session to continue where you left off or to start from instructions that are known to work well.

Prerequisites

- You have configured a playground for your project.
- The MLflow service is available in your project.
- One or more prompts have been saved in the project.

Procedure

1. From the OpenShift AI dashboard, click **Gen AI studio → Playground**.
2. In the chatbot settings panel, click the **Prompt** tab.
3. Click **Load Prompt**.
The **Load prompt** dialog opens, displaying a table of saved prompts in the current project.
4. Optional: Use the search field to filter prompts by name prefix.
5. Click a prompt in the table to view its versions in the side panel.
6. Optional: From the **Version** dropdown, select a specific version.
7. Click **Load in Playground**.
The selected prompt version is loaded as the system instructions for the current chatbot configuration.

Verification

- The prompt name and version number are displayed in the **Prompt** tab.
- The **Instructions** field contains the text from the loaded prompt version.

CHAPTER 10. TESTING WITH MODEL CONTEXT PROTOCOL (MCP) SERVERS

Authorize and interact with connected MCP servers to use their integrated tools directly from the playground chat.

Prerequisites

- You have deployed a model with tool-calling capabilities enabled in your project.
- You have configured a playground instance for your project.
- A cluster administrator has configured an MCP server, and the server is listed and available in the **MCP** tab of the settings panel.

Procedure

1. In the settings panel, click the **MCP** tab.
2. Select the checkbox for the server that you want to use.
3. Click the **Auth** icon next to the server name.
The **Authorize MCP server** dialog opens.
4. If the server requires a token, enter the token in the **Access token** field and click **Authorize**.
A **Connection successful** message appears.



NOTE

Authorization tokens for MCP servers are stored only for the current browser session. If you close your browser, you must re-authorize the server.

5. Click **Close**.
6. Click the **View tools** (wrench) icon for the same MCP server.
A modal appears, listing all available tools for that server. You can copy a tool name to use in the chat.
7. In the chat input field, type a query that uses one of the available tools.
8. Click the **Send** button or press Enter.



NOTE

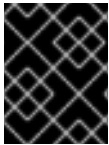
After you send a prompt, the **Send** button changes to a **Stop** button. Click it to interrupt the model response.

Verification

- The AI bot responds, indicating it is using the tool.
- The bot provides the output from the tool.

CHAPTER 11. EXPORTING YOUR PLAYGROUND CONFIGURATION

Export your gen AI playground configuration as a Python code template so that you can use it in your local development environment, such as a notebook or IDE.



IMPORTANT

This code is a template and is not a runnable script. It provides a starting point that shows your configuration, including the model, MCP tools, and RAG files used.

Prerequisites

- You have configured your playground instance with the settings that you want to capture in your code template. This includes:
 - Selecting a model.
 - Setting model parameters, such as model temperature, to your desired values.
 - Optional: Uploading files or selecting a vector store in the **Knowledge** tab.
 - Optional: Authorizing and enabling servers in the **MCP** tab.

Procedure

1. In the playground, configure your desired settings.
2. Click the **View code** button.
A dialog opens, displaying a Python code template.
3. Click **Copy code**.
4. Paste the code into your local development environment.

Verification

- Review the pasted code in your local environment.
- Confirm that the template includes the correct model, MCP tools, and RAG files from your playground configuration.

CHAPTER 12. UPDATING YOUR PLAYGROUND CONFIGURATION

You can update the configuration of your playground instance to add new models, re-register models that were stopped, or change the existing configuration.



WARNING

Updating the playground configuration will permanently delete the inline vector database for **all users** in your project.

Prerequisites

- You have configured a playground for your project.

Procedure

1. From the OpenShift AI dashboard, click **Gen AI studio** → **Playground**.
2. Select the project containing your model deployment from the **Project** dropdown list.
3. In the upper-right corner of your playground, click the action menu (⋮) and select **Update configuration**.
4. On the configuration screen, select or clear the checkboxes for the models you want to make available.
5. Click **Update**.

Verification

- The playground configuration is updated with a new selection of models.

CHAPTER 13. DELETING A PLAYGROUND FROM YOUR PROJECT

You can delete a playground instance from a project. This removes instance for all users who have access to that project.

Prerequisites

- You have configured a playground for your project.

Procedure

1. From the OpenShift AI dashboard, click **Gen AI studio → Playground**.
2. Select the project containing your model deployment from the **Project** drop-down list.
3. In the upper-right corner of your playground, click the action menu (⋮) and select **Delete playground**.



NOTE

This action deletes the playground for **every user** in the project.

4. Confirm the deletion.

Verification

- Confirm that the playground is deleted from the project.
- In the **Gen AI studio → AI asset endpoints** page, models no longer show the **Try in playground** button and instead show the **Add to playground** button.

CHAPTER 14. NEXT STEPS

You have successfully deployed and tested a model using the playground with RAG and MCP tools. For more information on the next steps, see the following resources:

Developing in an IDE

[Working in your data science IDE](#)

Learn how to access your workbench IDE (JupyterLab, code-server, or RStudio Server) to develop models.

CHAPTER 15. TROUBLESHOOTING PLAYGROUND ISSUES

If you encounter issues while using the playground, refer to the following scenarios and solutions.

15.1. THE CHATBOT THINKS INDEFINITELY

Problem After sending a query, the chatbot shows a thinking indicator but never returns a response.

Cause This issue often occurs when the query or the accumulated context exceeds the maximum context length (sequence length) configured for the model.

Solution

1. In the OpenShift AI dashboard, click the **Applications** menu and select **OpenShift Console**.
2. Navigate to your project's namespace.
3. Check the logs for the following pods:
 - The playground pod: **lsd-genai-playground-`<id>`**
 - The model serving pod: **`<model-name>-predictor-<id>`**
4. Look for errors related to context length limits or memory (OOM) constraints.

15.2. THE MODEL DOES NOT USE RAG DATA

Problem The model answers questions using its training data instead of searching the uploaded RAG documents.

Solution

In the **Prompt** tab of the settings panel, update the system instructions to explicitly force the use of the search tool.

- **Example:** "You MUST use the **knowledge_search** tool to obtain updated information."
- **Example:** "Always search the knowledge base before answering questions about company policies."

15.3. MCP SERVERS ARE MISSING FROM THE UI

Problem The **MCP** tab is empty or not visible in the settings panel.

Cause MCP servers must be configured at the cluster level by an administrator.

Solution

Contact your OpenShift AI administrator to configure the required MCP servers. Administrators can find a list of available servers in the Red Hat OpenShift AI documentation.

15.4. THE MODEL FAILS TO CALL MCP TOOLS

Problem The model attempts to use a tool but fails, or outputs raw XML tags (e.g., **`<tool_call>`**).

Cause

- The model does not support tool calling.
- The vLLM runtime arguments are missing or incorrect.
- **Known Issue:** Some models (e.g., **Qwen3-4B-Instruct**) may output raw tags if the correct reasoning parser is not available in the current vLLM version.

Solution

1. Verify the model supports tool calling on its Hugging Face model card.
2. In the model's deployment settings, ensure the following **Custom Runtime Arguments** are present:
 - **--enable-auto-tool-choice**
 - **--tool-call-parser**
3. If the model outputs **<think>** tags, you can hide them by adding **/no_think** to your prompt.